

观澜视频平台技术总结报告

观澜视频平台 · 第 4 组 · 2026 夏季软件工程基础实践

可编辑源文件与本 PDF 同时提交；本文由仓库脚本生成，页脚标注页码，最终以 Git 中的 Markdown/Mermaid 源为准。

观澜视频平台技术总结报告

■ 交付状态：DEL-01 = **HUMAN EVIDENCE PENDING**。本报告是 **06_defense** 唯一的权威技术汇总源，并替代原 **technical-summary.md**。仓库技术材料已具备可复核的合并条件；教师确认、ARCH 会议原件、成员签字、非作者 clean-machine 复现、备用录屏和全员计时演练等真人证据尚未补齐，故 DEL-01 不得标记为完成或关闭。

1. 目标与范围

1.1 项目目标

观澜视频平台是一个面向 Web 的在线视频、直播和内容治理系统。项目原始形态为前后端分离的 NestJS 单体应用，本阶段在保留原系统可运行基线的前提下，完成了面向课程验收的微服务、部署、测试和证据建设。

本项目的目标不是只增加服务目录，而是完成以下技术闭环：

1. 固定改造前的单体版本，保证原系统可以启动、测试和回退。
2. 将业务拆分为至少三个可独立构建、测试和部署的业务服务。
3. 为每个业务数据模型指定唯一 owner，禁止跨服务直接查询或写表。
4. 通过 API Gateway 统一入口、路由、服务身份和阶段性切流。
5. 为跨服务请求提供 requestId、服务 JWT、timeout、幂等和失败处理。
6. 为各服务建立独立 Prisma schema、migration、数据库账号和容器运行方式。
7. 通过单元测试、API/集成测试、浏览器 E2E 和双目标回归验证业务闭环。
8. 通过 HPA、依赖故障恢复和性能对比实验展示系统的弹性和改造代价。

改造前单体基线使用 annotated tag **monolith-start** 固定。

1.2 最终业务范围

经过范围冻结，最终验收采用六条端到端业务用例。页面、单个接口或一次数据库操作不单独作为业务用例；每个用例都需要具备参与者、触发条件、前置条件、主成功流程、异常/备选流程和可验证结果。

编号	用例	参与者	最终汇报
UC01	账户访问与资料维护	游客、普通用户、管理员	
UC02	视频发现、播放与观看记录	游客、普通用户	
UC03	投稿、媒体处理、审核与发布	创作者、管理员、MinIO、FFmpeg	重点讲解
UC04	视频互动、关注与通知	普通用户、创作者	
UC05	直播、互动、礼物与录播回看	主播、观众、SRS、MinIO	重点讲解
UC06	举报、审核与治理处置	普通用户、管理员、AI 辅助服务	重点讲解

1.3 六个业务用例的技术说明

UC01：账户访问与资料维护

- 参与者：游客、普通用户、管理员。
- 触发条件：用户注册、登录、修改资料，或访问需要权限的功能。
- 前置条件：后端、数据库和验证码服务可用；普通用户、创作者和管理员账号已经准备。

- 主流程：用户进入注册/登录页，系统校验验证码和账号密码，创建或查询用户并签发凭据；登录后根据角色展示对应页面；用户修改昵称、头像或简介；刷新页面或重新登录后验证资料仍然存在。
- 异常流程：验证码错误、用户名或邮箱重复、密码错误、会话过期、普通用户访问管理员接口、管理员入口密钥错误。
- 技术实现：单体中由 **auth**、**captcha**、**email**、**user** 模块完成；微服务中由 **identity-community** 管理 User、Profile、Follow、Message、Notification 等身份和社区数据；Gateway 负责将外部身份转换为可信的服务 JWT。
- 验证证据：UNIT-TC01、INT-TC01、E2E-TC01、REG-01 的 UC01 结果。

UC02：视频发现、播放与观看记录

- 参与者：游客、普通用户。
- 触发条件：用户从推荐、分区、搜索或关注流中寻找视频并播放。
- 前置条件：数据库存在已发布视频，媒体资源可访问，推荐和搜索接口正常。
- 主流程：用户进入首页、分区或搜索页；系统只返回公开发布内容；用户进入详情页，加载媒体、评论和相关推荐；播放过程中上报观看进度；登录用户在历史记录中看到本次观看。
- 异常流程：搜索无结果、草稿或驳回内容不可见、媒体代理失败、视频资源不可达、游客不记录个人历史、重复进度上报。
- 技术实现：**feed**、**search**、**video**、**media-proxy** 和 **content-media** 负责内容发现和媒体读取；Video 的发布状态是公开可见性的主要约束；观看记录以外部 **userId** 关联 **identity** 用户，不在 **content schema** 建立跨服务外键。
- 验证证据：UNIT-TC02、INT-TC02、E2E-TC02、REG-01 的 UC02 结果。

UC03：投稿、媒体处理、审核与发布

- 参与者：创作者、管理员、MinIO、FFmpeg。
- 触发条件：创作者希望上传作品并公开发布。
- 前置条件：创作者已登录；MinIO、MySQL 和媒体处理能力可用；管理员账号存在。
- 主流程：创作者选择视频和封面，填写标题、简介和分区；**content-media** 接收 **multipart** 文件并校验类型、大小、扩展名；原始对象写入 MinIO，**VideoAsset** 登记 **objectKey** 和媒体元数据；**FFprobe** 验证上传文件包含视频流；稿件保存为草稿并提交审核；治理服务创建审核任务；管理员审核通过后，**content-media** 将视频置为 **PUBLISHED**，视频进入推荐、搜索和播放范围。
- 异常流程：上传超时、非法媒体、MinIO 不可用、**FFprobe** 验证失败、重复提交、治理服务不可用、管理员驳回、创作者修改后重新提交。
- 技术实现：**content-media** 是 **Video**、**VideoAsset**、**VideoCategory**、互动数据和创作者内容统计的 **owner**；**governance-ai** 只保存 **VideoReview** 和审核决定，通过 **POST /internal/v1/videos/:id/review-decision** 应用状态，不直接写 **content** 表；提审和审核决定分别使用 **requestId/decisionId** 保证幂等。
- 验证证据：UNIT-TC03、INT-TC03、E2E-TC03、**scripts/content-publishing-smoke.mjs** 和 REG-01 的 UC03 结果。
- 最终汇报重点：该用例展示对象存储、上传媒体校验、稿件状态机、跨角色审核、跨服务调用和失败恢复。整体工程环境安装 **FFmpeg** 并在测试媒体生成等场景使用；当前 **services-mode** 的 **content-media** 运行路径只实现 **FFprobe** 视频流验证，未将 **FFmpeg** 转码或抽帧作为已交付的服务能力。

UC04：视频互动、关注与通知

- 参与者：普通用户、创作者。
- 触发条件：用户希望点赞、收藏、评论、回复、发送弹幕或关注创作者。
- 前置条件：用户已登录；目标视频或用户存在且允许互动。

- 主流程：用户在视频详情页执行互动；系统保存关系或内容，使用唯一约束/receipt 防止重复写入；更新展示计数；通过 NotificationOutbox 向被互动用户创建通知；用户在收藏夹、关注流或通知页验证结果。
- 异常流程：游客操作、重复点赞/收藏/关注、目标不存在、违规文本、评论撤回、通知服务失败、取消操作。
- 技术实现：VideoLike、Favorite、FavoriteFolder、VideoDanmaku、Comment 等归 content-media；FollowRelation、Notification 等归 identity-community；主体操作和通知投递解耦，通知失败不会回滚已经成功的点赞或评论；跨服务通知使用 requestId 幂等和有限重试。
- 验证证据：UNIT-TC04、INT-TC04、E2E-TC04、scripts/content-interaction-smoke.mjs 和 REG-01 的 UC04 结果。

UC05：直播、互动、礼物与录播回看

- 参与者：主播、观众、SRS、MinIO。
- 触发条件：主播创建直播间并开播，观众进入观看并互动，直播结束后回看录播。
- 前置条件：主播和观众账号存在；SRS、MinIO、数据库可用；浏览器允许摄像头、麦克风或屏幕共享。
- 主流程：主播创建直播间；系统生成推流和播放信息；主播开始直播；观众从直播广场进入房间并建立播放/信令连接；观众发送聊天、礼物或其他互动；主播停止直播；live-reward 保存 Session 结束状态并通过 content 内部接口登记录播资产；录播完成后可回看。
- 异常流程：SRS 不可用、浏览器权限拒绝、主播断连、观众重连、余额不足、重复扣款、录播登记失败、服务重启。
- 技术实现：live-reward 管理 LiveRoom、LiveSession、LiveMessage、LiveViewerEvent、ReplayRegistration、CoinTransaction、DailyCoinClaim、StreakMilestoneClaim 和 VideoCoinContribution；SRS 只承担推流/播放基础能力，业务状态和账本由服务数据库负责；录播登记使用 objectKey 和 requestId 幂等，失败时保留可重试或最终失败状态；普通直播消息按 7 天和单 Session 10,000 条规则管理，业务事实和审计事实不随消息清理。
- 验证证据：UNIT-TC05、INT-TC05、E2E-TC05、scripts/live-reward-content-smoke.mjs、REG-01 的 UC05 结果和 EXP-02 故障恢复结果。
- 最终汇报重点：该用例用于展示 SRS 外部依赖、实时状态、持久化恢复、回放登记、礼物币幂等和故障隔离。

UC06：举报、审核与治理处置

- 参与者：普通用户、管理员、AI 辅助服务。
- 触发条件：用户发现违规视频、评论或弹幕并发起举报，管理员需要处理。
- 前置条件：举报者已登录；举报目标存在；管理员权限和治理服务可用。
- 主流程：用户提交举报原因；governance-ai 校验目标并保存 ReportRecord 和目标快照；管理员查看举报详情和审核队列；管理员选择保留、隐藏或删除等治理动作并填写原因；治理服务保存 ModerationDecision、操作者、时间和 requestId；必要时通过 content 内部 API 应用目标内容状态；举报人收到处理通知。
- 异常流程：目标不存在、重复举报、普通用户访问管理接口、AI 辅助不可用、content 状态更新失败、重复处置、处置重试。
- 技术实现：governance-ai 是 VideoReview、CommentAiTask、ReportRecord、ModerationDecision 的 owner；举报保存目标类型、目标 ID 和快照，避免下游暂时不可用时丢失举报事实；治理服务通过 [POST /internal/v1/videos/:id/text-status](#) 或审核决定接口改变内容状态；content 不可用时决定进入 APPLY_PENDING，由补偿任务重试。
- 验证证据：UNIT-TC06、INT-TC06、E2E-TC06、治理 smoke、REG-01 的 UC06 结果。
- 最终汇报重点：该用例用于展示权限、审计、状态处置、补偿和跨服务一致性。

2. 架构结果

2.1 总体架构

系统采用浏览器/服务器架构，前端使用 Vue 3 + TypeScript，单体后端使用 NestJS + Prisma，微服务模式通过 API Gateway 对外提供统一入口。

代码 / 模型源 (text)

```
浏览器
-> Vue 3 SPA
-> API Gateway
  -> identity-community
  -> content-media
  -> live-reward
  -> governance-ai
  -> monolith fallback
-> MySQL / Redis / MinIO / FFmpeg / SRS
```

Gateway 支持 **monolith** 和 **services** 两种路由模式。未开放的能力继续转发到单体；已开放能力通过读写 allowlist 进入对微服务。这样可以分阶段验证，而不是在所有服务同时切换后再定位问题。

运行时主要端口如下：

组件	端口	作用
Vue 前端	5173	浏览器访问
单体 NestJS	3000	单体 API
Gateway	3100	微服务统一入口
identity-community	3101	身份/社区服务
content-media	3102	内容/媒体服务
live-reward	3103	直播/礼物服务
governance-ai	3104	治理服务
MySQL	3306	结构化数据
Redis	6379	缓存和辅助状态
MinIO API/Console	9000/9001	对象存储
SRS HTTP/API	8080/1985	播放和管理 API
SRS RTMP/WebRTC	1935/8000 UDP	推流和实时播放

2.2 服务职责与数据归属

服务	负责内容	不负责内容
identity-community	账号、资料、关注、私信、通知、动态社区、用户摘要	视频状态、媒体对象、直播房间、最终审核决定
content-media	视频、视频资产、分类、推荐搜索、观看记录、评论、点赞、收藏、弹幕、创作者统计	用户账号、直播 Session、礼物账本、最终治理决定
live-reward	直播房间、Session、观众、消息、SRS 协同、录播登记、礼物币和奖励账本	用户资料、视频主记录、审核决定
governance-ai	视频审核、文本审核、举报、处置、审计、AI 辅助任务	直接写 content、identity、live 数据表
gateway	路由、服务身份、requestId、fallback、切流和回滚	业务数据和业务表

当前 31 个 Prisma Model 均已分配唯一 owner。跨域字段保存外部 ID，不建立跨 schema 外键；需要昵称、标题或内容快照时通过内部 API、事件快照或缓存获取。具体 31/31 归属见 [docs/practice-2026/08-service-boundaries-and-data-ownership.md](#)。

2.3 服务接口和跨服务调用

公共服务 contract 统一约束响应 envelope：

```
{
  "code": 0,
  "message": "ok",
  "data": {},
  "requestId": "..."
}
```

主要内部接口包括：

调用方 -> 被调用方	接口	用途
content -> identity	POST /internal/v1/users/batch-summary	批量获取昵称和头像摘要
content -> identity	POST /internal/v1/notifications	创建互动通知
content -> governance	POST /internal/v1/reviews	创建视频或文本审核任务
governance -> content	POST /internal/v1/videos/:id/review-decision	应用审核决定
governance -> content	POST /internal/v1/videos/:id/text-status	隐藏、恢复或删除文本目标
live -> content	POST /internal/v1/replays	登记直播录播资产
live -> content	视频代币相关内部 contract	应用代币展示计数
live/governance -> identity	通知内部 contract	发送直播、举报和治理通知

内部请求使用服务账号 JWT。JWT 包含调用方、受众、scope 和过期时间，密钥通过环境变量或 Kubernetes Secret 注入，不写入仓库，也不在日志中打印完整 Token。

2.4 跨服务 timeout、重试和失败结果

业务场景	调用链	timeout/重试	失败时结果
提交审核	content -> governance	2 秒；网络错误最多重试 2 次；requestId 幂等	提审接受后稿件为 PENDING_REVIEW ；上游结果不确定时保留该状态并使用相同 requestId 重试，不直接发布；只有明确拒绝时才恢复原 DRAFT 或 REJECTED 状态
应用审核决定	governance -> content	2 秒；decisionId 幂等重试	决定保持 APPLY_PENDING ，后台可继续补偿
创建通知	content -> identity	1 秒；异步有限重试	主体点赞/评论成功，通知进入待补偿
用户摘要	content -> identity	1 秒；只读重试 1 次	使用短期快照或显示用户信息暂不可用
录播登记	live -> content	5 秒；指数退避；objectKey 幂等	直播正常结束，回放显示处理中
SRS 调用	live -> SRS	2 秒；不进行无限重试	开播/观看返回降级提示，其他服务保持健康
举报目标摘要	governance -> content	1 秒；只读重试 1 次	先保存目标 ID、类型和快照
应用治理处置	governance -> content	2 秒；decisionId 幂等	保留审核决定，目标状态显示 APPLY_PENDING

3. 数据迁移与一致性

3.1 迁移设计

四个业务服务分别维护独立的 Prisma schema、migration 和运行时数据库账号。数据库可以位于同一 MySQL 实例，但通过不同 database/schema 和最小权限账号隔离。

迁移安全策略包括：

- source 和 target 必须通过一次性确认值授权，source=target 直接拒绝；
- 默认只允许明确命名的测试库或测试 schema；
- 非测试目标要求精确 host、port 和 database 授权；
- 迁移脚本可重复执行，重复执行不产生结构漂移；
- 迁移后比较表数量、字段、唯一约束、行数和边界数据；
- 发现目标多余、缺失或字段不一致立即停止；
- 不删除单体 owner 表，不在共享或生产环境执行 reset。

主要迁移入口：

- [scripts/identity-cutover-migrate.mjs](#)
- [scripts/content-cutover-migrate.mjs](#)
- [scripts/live-cutover-migrate.mjs](#)
- [scripts/governance-cutover-migrate.mjs](#)
- [scripts/db-migrate.sh](#)
- 各服务 Prisma migration 目录

3.2 迁移顺序

1. 固定 **monolith-start**，保留原 Prisma schema、migration 和 seed。
2. 为 identity、content、live、governance 建立独立 schema 和数据库账号。
3. 先实现 identity-community 和 content-media 的只读接口。
4. 迁移用户/关系和视频/内容数据，校验行数、唯一约束和抽样结果。
5. 将跨域外键改为普通外部 ID，增加批量摘要和内部状态 API。
6. 将直播房间、Session、观众、消息和录播登记从进程内状态转为数据库事实。
7. 提取 governance-ai，通过内部 API 应用审核和治理决定。
8. 在相同 Seed 数据上运行 UC01-UC06 双目标回归。
9. 回归通过后再停止对应单体写流量，并保留 Gateway monolith rollback。

3.3 一致性和幂等

- 点赞、收藏、关注使用唯一约束或 upsert，重复操作返回可解释结果。
- 投稿、提审、撤回、审核决定和举报使用 requestId 或 decisionId 防止重放。
- 礼物币和投币使用 live-reward 账本作为唯一事实，余额更新在事务中完成。
- 通知使用 NotificationOutbox，主体操作和通知投递解耦。
- 录播使用 objectKey、sessionId 和状态字段控制重复登记。
- governance 保存目标快照，即使 content 暂时不可用也不丢失举报事实。
- 跨服务失败不依赖分布式事务，而是使用状态机、补偿任务、有限重试和显式 APPLY_PENDING 状态。

4. 六 UC 回归与测试结果

4.1 测试分层

层级	目标	主要入口
需求/单元测试	验证方法、业务规则、异常分支和状态转换	<code>npm run test:requirements</code> 、Jest、Vitest
API/集成测试	验证模块间调用、数据库、外部依赖 contract	<code>npm run test:api</code> 、各 service test
浏览器 E2E	从页面交互验证真实用户链路	<code>npm run test:e2e</code>
REG-01 双目标回归	单体和 Gateway 各执行 UC01-UC06	<code>npm run reg:01</code>
工程实验	验证 HPA、故障恢复和性能差异	HPA、fault、performance 脚本

REG-01 v2 为两个目标分别创建隔离 creator、actor 和媒体数据，按照账户、内容发现、投稿审核、互动通知、直播录播、举报治理的依赖顺序执行用例。每个目标输出公开 endpoint 清单以及 **PASS/FAIL/BLOCKED/NOT RUN**；严格模式下任一业务失败或未完成用例都会使命令非零退出。

4.2 最终回归结论

标准 Compose 同时启动四个业务数据库、MinIO、SRS、四个业务服务、Gateway，以及用于单体对照的独立数据库和后端。同一 runner 对单体和微服务 Gateway 各执行六个用例：

- 单体：6/6 PASS；
- 微服务 Gateway：6/6 PASS；
- 合计：12/12 PASS；
- 回归结束：monolith rollback PASS；
- 临时容器、数据库、端口和测试数据：按范围清理 PASS。

最终原始报告包包括：

- `delivery/04_tests/raw/github-run-33379394312/run.json`
- `delivery/04_tests/raw/github-run-33379394312/job-logs/`
- `delivery/04_tests/raw/github-run-33379394312/artifacts/`
- `delivery/04_tests/raw/github-run-33379394312/experiments/`
- `delivery/04_tests/raw/github-run-33379394312/checksums.sha256`

4.3 代表性技术断言

- UC01：角色权限、资料持久化、管理员入口和未授权访问均有断言。
- UC02：仅允许 PUBLISHED 内容出现在推荐、搜索和详情公开路径。
- UC03：非法媒体被拒绝；合法 MP4 完成 MinIO、FFprobe、VideoAsset 和稿件流程；审核前不公开。
- UC04：重复点赞/收藏/关注幂等；通知失败不回滚主体互动。
- UC05：直播房间、Session、观众、消息、账本和录播登记可查询；SRS 故障返回明确失败。
- UC06：举报、审核、处置通知、重复处置拒绝和目标状态回写可验证。

5. CI/CD 与部署

5.1 构建和测试流水线

GitHub Actions 和 Jenkins 均采用分阶段门禁：

```

安装依赖
-> lint
-> build
-> requirements/unit/API tests
-> public E2E
-> Git SHA 镜像
-> Kind migration/deploy
-> health/version/0 restart
-> Artifact and cleanup

```

`npm run test:ci` 依次生成 Prisma Client，执行单体、前端和六个 workspace 的 lint/build/test，以及 REG harness。CI 中的失败会停止后续镜像和部署阶段。

Jenkins 额外验证了：

- 正常成功流水线；
- Unit 阶段故意失败并阻断后续阶段；
- SCM 自动触发；
- 正式 migration；
- JUnit XML 发布；
- 失败和成功日志、Artifact、cleanup 均可追溯。

5.2 容器化和 Kubernetes

单体和微服务均提供 Dockerfile。微服务 Compose 使用独立的 identity、content、live-reward 和 governance MySQL，以及 MinIO、SRS、四个服务和 Gateway。每个服务均提供：

- `/health/live`
- `/health/ready`
- `/version`

Kubernetes 部署包括：

- Namespace、ConfigMap、Secret；
- 四个 migration Job；
- 五个 Deployment：Gateway 和四个业务服务；
- Service、探针、资源 request/limit；
- MinIO StatefulSet/PVC；
- HPA；
- 健康检查和资源清理脚本。

镜像使用 Git SHA tag，不使用不可追溯的 `latest` 作为验收版本。数据库密码、服务 JWT、MinIO Secret 和管理员密钥通过环境或 Kubernetes Secret 注入。

5.3 部署回滚步骤

部署或切流失败时执行：

1. 停止新写流量，记录失败请求和 requestId。
2. 将 Gateway 的路由切回 `GATEWAY_ROUTE_MODE=monolith`。
3. 或者从 read/write allowlist 中移除目标服务。
4. 检查单体、Gateway、业务服务的 live/ready/version。

5. 运行对应代表性 UC 和回归探针。
6. 保存迁移比较、日志、错误响应和失败资源。
7. 不删除单体表，不对共享库执行 reset。
8. 修复后重新执行 migration、contract、Compose Gate 和 REG-01。
9. 通过后再按 identity -> content -> live -> governance 分阶段恢复切流。

6. 弹性、故障与性能

6.1 HPA 自动扩缩容实验

实验脚本：[scripts/hpa-experiment.sh](#)。

实验配置：

- 目标：Kubernetes 中的 Gateway Deployment；
- HPA：[autoscaling/v2](#)；
- minReplicas：1；
- maxReplicas：3；
- CPU averageUtilization：25%；
- Gateway CPU request/limit：50m/500m；
- 负载：64 个 Node workers；
- 负载持续时间：最多 120 秒；
- 扩容策略：15 秒内最多增加 2 个 Pod；
- 缩容策略：30 秒稳定窗口，15 秒内最多减少 1 个 Pod；
- 指标：官方 metrics-server，测试集群追加 [--kubelet-insecure-tls](#)，生产 manifest 不使用该测试参数。

关键时间线：

时间 (UTC)	阶段	Ready/Desired	CPU	结论
06:54:15	baseline	1/1	2%	基线稳定
06:54:31	load	1/3	104%	HPA 请求最大副本数
06:54:36	load	3/3	104%	扩容完成，约 21 秒
06:55:46	recovery	3/3	2%	CPU 恢复
06:56:01	recovery	2/2	2%	第一段缩容
06:56:16	recovery	1/1	2%	缩容完成，约 30 秒

实验结论：真实 HPA controller 完成了 1 -> 3 -> 2 -> 1 的扩缩容，而不是通过脚本直接手工 scale。实验结束后删除负载 Pod、HPA 和临时 metrics-server 资源。

6.2 依赖故障恢复实验

实验脚本：[scripts/fault-experiment-probe.mjs](#)。

依赖	故障操作	受影响结果	未受影响结果	恢复结果
live MySQL	停止 live-mysql	live ready 503	identity/content/governance/Gateway 200	live ready 200
SRS	将 SRS_API_BASE 指向 127.0.0.1:1	room start 503	identity/governance/Gateway 200	同一房间 start 200，stop 后为 ENDED

依赖	故障操作	受影响结果	未受影响结果	恢复结果
MinIO	停止 <code>content-minio</code>	合法上传 500	identity/live/governance/Gateway 200	合法上传 200，assetId/uploadToken 有效

该实验验证了故障域隔离：受影响的业务按照 contract 明确失败，其他服务不会跟随崩溃，依赖恢复后原业务路径能够恢复。实验没有把外部依赖不可用时“所有业务继续成功”作为目标。

6.3 单体/微服务三接口性能对比

实验脚本：`scripts/performance-compare.mjs`，报告 schema 为 `perf-01/v2`。

实验方法：

- 两个目标在同一 Compose 运行期间、同一 host 上执行；单体为 `http://127.0.0.1:3200`，Gateway 为 `http://127.0.0.1:3100`；
- 三条业务路径分别为推荐流、搜索和视频详情；
- 两个隔离数据库均只保留同一条 `id=1` 的 PUBLISHED 视频，计时前三接口必须返回相同 `id:title` 业务签名；
- 每接口、每目标预热 20 次；正式执行 3 轮，每轮 240 请求，并发 16，读取完整响应体，timeout 5 秒；
- 轮次旋转接口顺序并交叉单体/Gateway 先后，避免固定热机与先后偏差；
- Gate：每接口/目标恰好 3 轮，错误数为 0，单轮 p95 不超过 1000 ms；不把“微服务必须快于单体”设为虚假门槛。

逐接口聚合结果：

记录 1

接口：推荐流

单体中位 p95：6.75 ms

微服务中位 p95：15.66 ms

p95 变化：+132.0%

单体中位 RPS：3009.32

微服务中位 RPS：1505.41

RPS 变化：-50.0%

结论：下降

记录 2

接口：搜索

单体中位 p95：24.29 ms

微服务中位 p95：19.20 ms

p95 变化：-21.0%

单体中位 RPS：935.16

微服务中位 RPS：1301.26

RPS 变化：+39.1%

结论：提升

记录 3

接口：视频详情

单体中位 p95：9.55 ms

微服务中位 p95：23.52 ms

p95 变化：+146.3%

单体中位 RPS：2320.96

微服务中位 RPS：961.41

RPS 变化：-58.6%

结论：下降

三接口、两目标、三轮合计完成 $3 \times 2 \times 3 \times 240 = 4320$ 次正式请求，全部返回 HTTP 200，0 error。搜索下沉到 content-media 后在本数据集上延迟和吞吐均提升；推荐流和详情页则受到 Gateway 与跨服务额外链路影响而下降。整体结论是 mixed 但偏下降，而不是“微服务必然提升性能”。该结果是同机、小并发、固定等价数据下的验收证据，不是生产容量承诺。

7. 工具、AI 使用和人工检查

7.1 工具及用途

工具	用途
Node.js、npm、TypeScript	运行时、workspace 管理、构建和脚本
Vue 3、Vite	前端 SPA 和开发服务
NestJS、Prisma、MySQL	单体后端、服务端模块、ORM 和数据库
Redis	缓存和辅助状态
MinIO	视频、封面、转码和录播对象存储
FFmpeg、FFprobe	转码、抽帧、时长检测和媒体校验
SRS	RTMP、HTTP 播放和 WebRTC 相关直播能力
Docker、Docker Compose	本地、CI 和隔离服务环境
Kubernetes、Kind、kubectl	服务部署、探针、迁移和 HPA
GitHub Actions、Jenkins	自动构建、测试、镜像、部署和 Artifact
Jest、Vitest、Supertest、Playwright	单元、服务、API/集成和浏览器测试
Mermaid	系统、组件、对象级模型源
ReportLab、Poppler	PDF 生成、渲染和版式检查
飞书云文档/Wiki	项目进度、看板和站会材料

7.2 AI 工具使用范围

生成式 AI 只用于辅助以下工作：

- 讨论服务边界、迁移顺序、回滚策略和实验方法；
- 草拟测试用例、异常分支和文档结构；
- 根据日志和失败断言提出故障定位方向；
- 整理需求、设计、测试和实验数据；
- 协助组织答辩 PPT 和技术总结的章节结构。

AI 输出不直接作为代码正确性、测试通过或实验结论。涉及业务规则、数据库 owner、接口 contract、迁移安全和性能数据的内容，必须以仓库代码、测试断言、原始日志和实际运行结果为准。

7.3 人工检查方式

- 所有代码改动由成员阅读 diff，并通过 Pull Request review 或 Owner review。
- 单元测试必须包含断言，不能只以进程退出码作为通过依据。
- API 和 E2E 测试必须验证状态码、响应结构、页面行为、数据库结果或状态变化。
- REG-01 明确区分 PASS、FAIL、BLOCKED 和 NOT RUN，不能把未执行项写成通过。
- 性能实验保存逐轮 CSV、参数、错误数和聚合计算，不能只提供手工平均值。
- HPA 必须观察真实 metrics-server、Pod 和 CPU 时间线，不能只根据 YAML 判定扩容成功。
- PDF 使用渲染结果检查页码、文字越界、空白页、表格和中文字体。
- 所有密码、Token、数据库口令、MinIO 密钥和个人隐私均通过环境变量或 Secret 管理，不写入仓库。

8. 技术结论与限制

8.1 主要技术结论

项目已经从可运行的单体基线推进到四个业务微服务和 Gateway 的完整技术闭环：

- 四个业务服务均有独立 schema、migration、数据库账号、镜像、探针和测试；
- 31 个 Prisma Model 均有唯一 owner；
- 跨服务调用使用 HTTP contract、服务 JWT、requestId、timeout、幂等和有限重试；
- identity、content、live、governance 均完成历史迁移、重复执行和数据一致性校验；
- Gateway 支持按能力切流，保留单体 fallback 和显式 rollback；
- 单体和 Gateway 各完成 UC01-UC06，合计 12/12 PASS；
- HPA 完成 1 -> 3 -> 2 -> 1 自动扩缩容；
- live MySQL、SRS、MinIO 三类依赖故障均完成失败和恢复验证；
- 性能三接口共完成 4320 请求，0 错误；搜索提升，推荐流和视频详情下降，所有 p95 低于 1000 ms；
- CI/CD 能在测试失败后阻断后续镜像和部署，并保存原始 Artifact。

8.2 当前限制

1. 性能实验是同机、低并发和单条等价发布数据的三接口验收实验，不代表生产容量。
2. SRS、MinIO、FFmpeg 和外部 AI 依赖具体环境；正式演示必须提前预检。
3. 单体表和入口仍保留，目的是降低迁移风险；这不代表已经完成生产环境的不可逆下线。
4. AI 只作为治理辅助，不自动执行封禁、删除或驳回，最终动作由管理员确认。
5. 教师确认原件、ARCH 会议原件、成员签字、非作者 clean-machine 复现、备用录屏和全员计时演练属于真人证据，不能由代码或 AI 自动生成。
6. 外部付费 AI 输出具有不确定性，测试只断言接口结构、模型标识、非空结果和持久化，不断言固定自然语言文本。

8.3 最终结论

本项目的核心成果不是简单拆分出四个服务，而是建立了一套可验证的工程机制：明确的数据归属、可重复的数据库迁移、可追踪的跨服务调用、可解释的失败结果、可执行的回滚路径和可复核的测试证据。

- 最终共识：DEL-01 仍为 **HUMAN EVIDENCE PENDING**。技术交付可进入合并与答辩准备，但在教师确认原件、ARCH 会议原件、成员签字与贡献核对、非作者 clean-machine 复现、实际备用录屏和全员计时演练全部补齐前，不得将 DEL-01 标记为 **DONE**、完成或关闭。